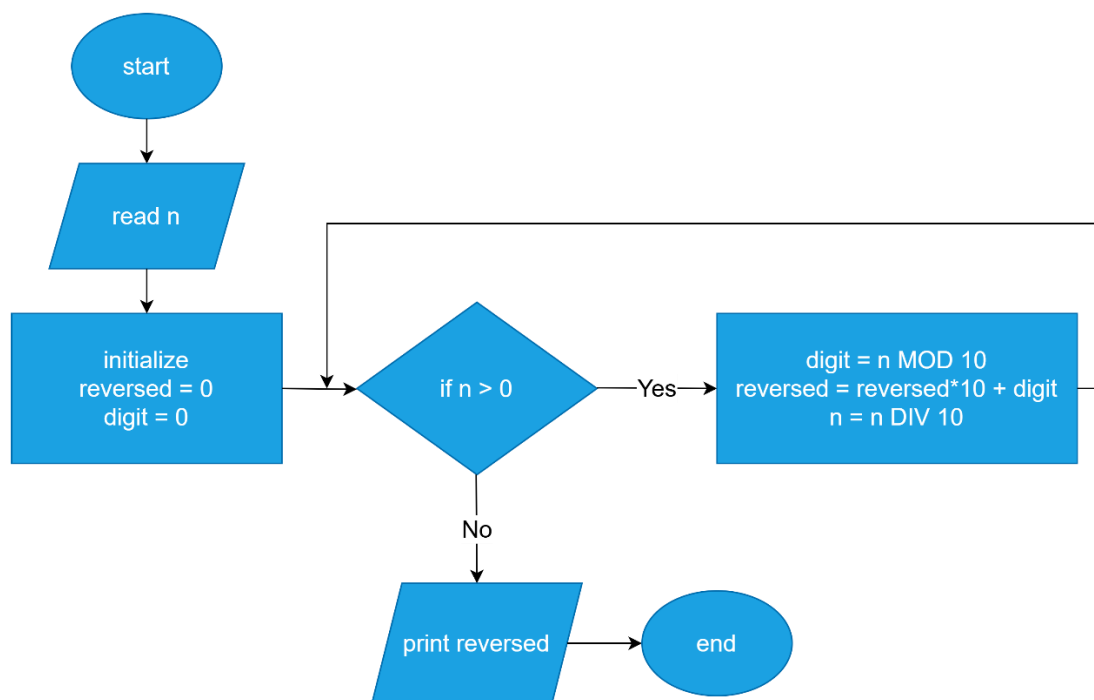




1. (15 نمره)

الف- (5 نمره) الگوریتمی را با استفاده از فلوجارت نمایش دهید که یک عدد صحیح مثبت را از کاربر دریافت کرده و مقلوب (برعکس) آن را محاسبه و چاپ نماید (مثلاً اگر ورودی ۱۲۳ بود، خروجی ۳۲۱ باشد).



ب- (10 نمره) با استفاده از الگوریتم بند الف، تابع زیر را به زبان C طوری بنویسید که یک عدد صحیح n را به عنوان آرگومان ورودی دریافت کرده و مقلوب آن را محاسبه کند و حاصل را در قالب مقدار بازگشتی برگرداند.

```
int Reverse(int n){  
  
}
```

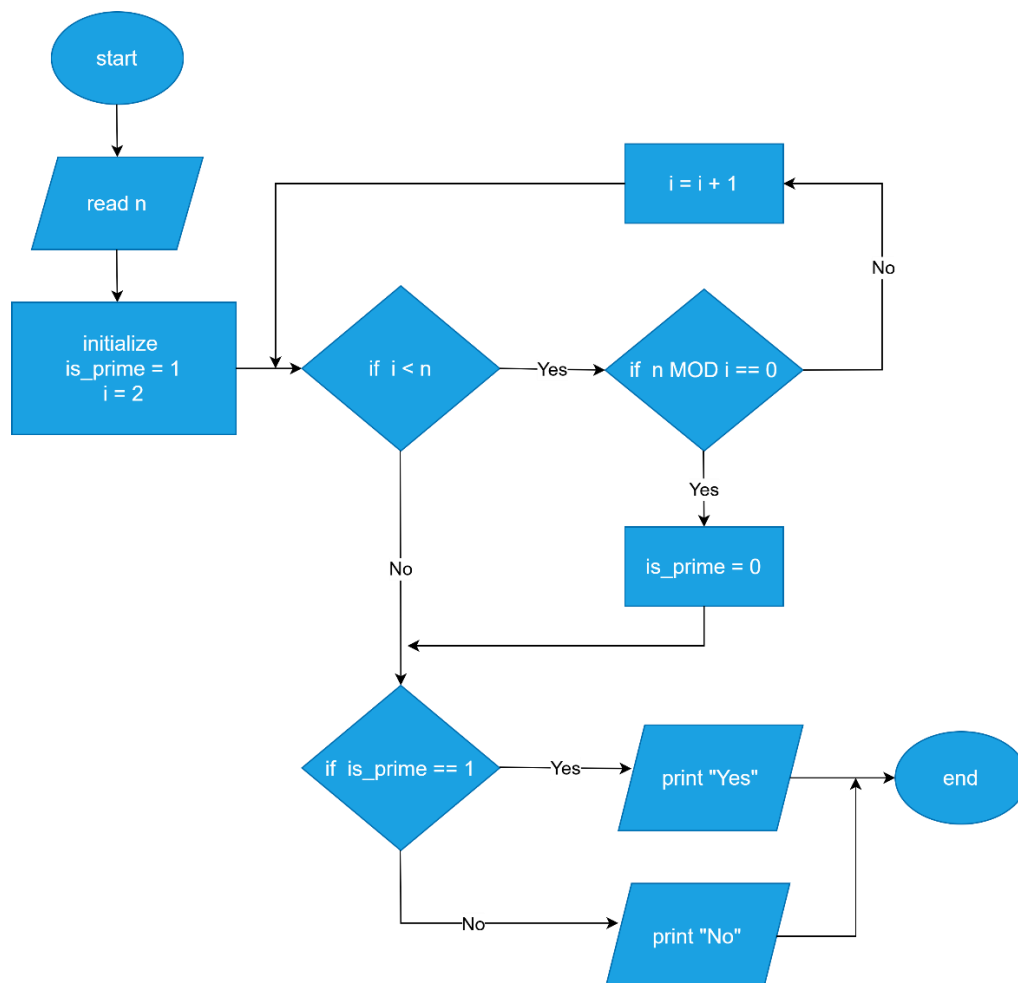
```

int Reverse(int n){
    int reversed = 0;
    int digit;
    while ( n>0 ){
        digit = n%10;
        reversed = reversed*10 + digit;
        n = n/10;
    }
    return reversed;
}

```

2. (25 نمره)

الف- (10 نمره) الگوریتمی را با استفاده از فلوچارت نمایش دهید که یک عدد صحیح مثبت ($n > 1$) را از کاربر دریافت کرده و بررسی کند آیا آن عدد اول است یا خیر. اگر اول بود پیام Yes و اگر نبود No چاپ کند.



(اگر n یک عدد مرکب باشد، آنگاه آن را می توان به صورت $n=a*b$ نوشت که حداقل یکی از a یا b کوچکتر یا مساوی $\sqrt{n}$ است. بنابراین اگر عددی مرکب باشد، حداقل یک مقسوم علیه کوچکتر یا مساوی $\sqrt{n}$ دارد؛ در غیر اینصورت عدد ما مرکب نیست و اول است. بنابراین می توانستیم به جای شرط $i < n$ بنویسیم $i * i \leq n$)

ب- (15 نمره) بر مبنای الگوریتم بند الف، تابع زیر را به زبان C طوری بنویسید که عدد n را به عنوان آرگومان ورودی دریافت کرده و اگر عدد اول است مقدار 1 و در غیر این صورت مقدار 0 را برگرداند.

```
int isPrime(int n){
}
```

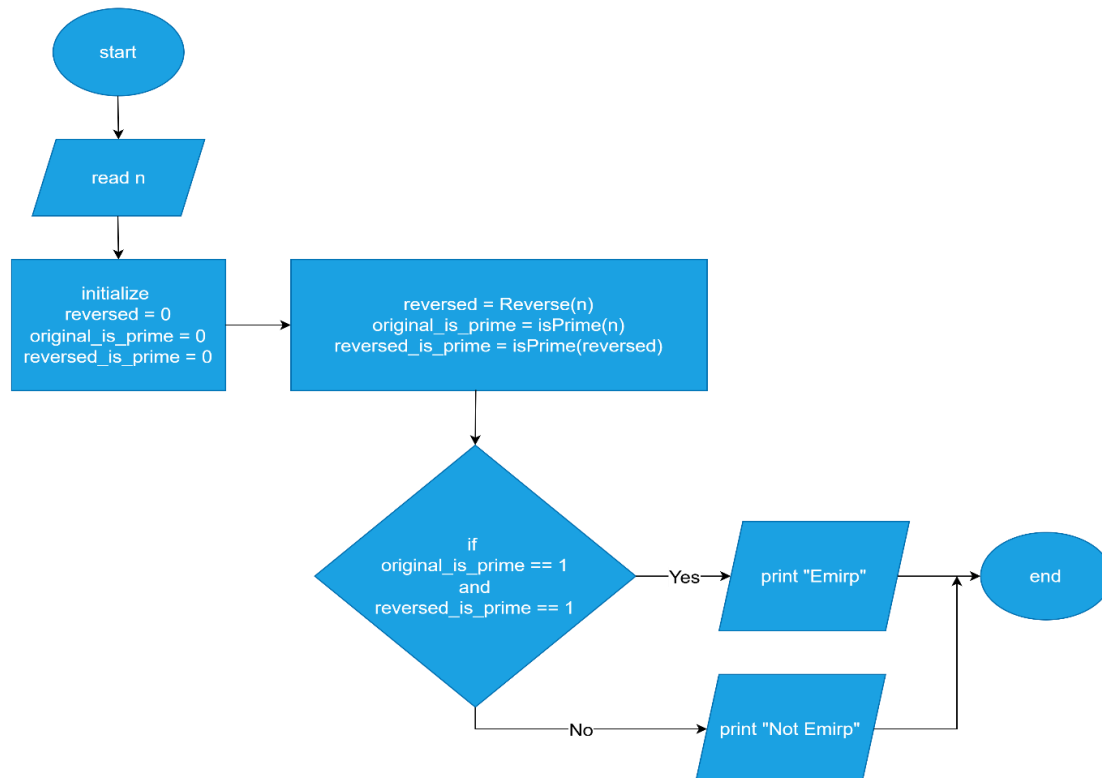
```
int isPrime(int n) {
    int is_prime = 1;
    for(int i=2 ; i<n ; i++){
        if(n % i == 0){
            is_prime = 0;
        }
    }
    if ( is_prime == 1 ){
        return 1;
    }
    else{
        return 0;
    }
}
```

با توجه به نکته مطرح شده در بند الف نیازی به بررسی بخش پذیر بودن اعداد تا n نیست و می توانیم تا $\sqrt{n}$ بررسی کنیم. همچنین با توجه به اینکه اگر عدد n بر عددی به غیر از یک و خودش بخش پذیر باشد، اول نیست و نیازی به بررسی اعداد بعد از آن نیست، پس می توان کد را به صورت زیر خلاصه و بهینه کرد:

```
int isPrime(int n) {
    for(int i=2 ; i*i<=n ; i++){
        if(n % i == 0){
            return 0;
        }
    }
    return 1;
}
```

3. (15 نمره) عددی Emirp است که هم خودش اول باشد و هم مقلوب (برعکس) آن عددی اول باشد (مثلاً 13 یک عدد Emirp است چون هم 13 اول است و هم 31)

الف- (5 نمره) این الگوریتم را با استفاده از دو تابع سوال‌های 1 و 2 در قالب یک فلوچارت بازنویسی کنید (فرض کنید توابع Reverse و IsPrime آماده هستند و فقط آن‌ها را فراخوانی کنید)



ب- (10 نمره) با استفاده از توابع سوال 1 و 2، تابع زیر را در زبان C طوری کامل کنید که مشخص کند عدد ورودی Emirp است یا خیر. در صورتی که عدد n ویژگی Emirp را داشت مقدار 1 و در غیر این صورت مقدار 0 را برگرداند.

```
int isEmirp(int n){
}
```

```
int isEmirp(int n) {
    int original_is_prime = isPrime(n);
    int reversed = Reverse(n);
    int reversed_is_prime = isPrime(reversed);

    if((original_is_prime == 1) && (reversed_is_prime == 1)){
        return 1;
    }
    return 0;
}
```

4. (15 نمره) دو ایراد در قطعه کد زیر وجود دارد. آنها را بیان و اصلاح کنید.

```
#include <stdio.h>
void calculateSum(int a, int b) {
    int total = a + b;
    return total;
}

int main() {
    int x, y;
    printf("Enter x and y: ");
    scanf("%d %d", &x, &y);

    int total = 0;
    calculateSum(x, y);
    printf("The Total is: %d", total);
    return 0;
}
```

الف) تابع `calculateSum` متغیر `total` که از جنس `int` است را برمی گرداند؛ اما در تعریف تابع، نوع مقدار بازگشتی تابع به صورت `void` تعریف شده است.

ب) در تابع `main`، متغیر `total` همواره صفر است و مقدار بازگشتی تابع `calculateSum` در هیچ متغیری ذخیره نمی شود. بنابراین برای اینکه کارکرد کد اصلاح شود، می بایست خروجی تابع را در متغیر `total` ذخیره کرد.

بنابراین کد اصلاح شده به صورت زیر است:

```
#include <stdio.h>
int calculateSum(int a, int b) {
    int total = a + b;
    return total;
}

int main() {
    int x, y;
    printf("Enter x and y: ");
    scanf("%d %d", &x, &y);

    int total = 0;
    total = calculateSum(x, y);
    printf("The Total is: %d", total);
    return 0;
}
```

5. (20 نمره) امتیازی**

در این مسئله، کاربر یک مقدار پولی را به عنوان ورودی وارد می کند. ما به دنبال آن هستیم تا ببینیم به چند طریق میتوانیم با استفاده از سکه های 2 و 5 و 7 تومانی، مقدار پول وارد شده توسط کاربر را با 2 شرط زیر بسازیم:

شرط 1) از هر نوع سکه باید حداقل 1 بار استفاده شود.

شرط 2) تعداد سکه های 2 تومانی استفاده شده باید زوج باشد.

در نهایت تعداد راه های ایجاد این مقدار پول را به عنوان خروجی به کاربر تحویل دهیم.

الف) (10 نمره) شبه کدی بنویسید که از کاربر یک مقداری برای پول به عنوان ورودی بگیرد و تعداد تمام حالات شرایط مسئله را به عنوان خروجی چاپ کند.

ب) (10 نمره) این تابع را در زبان C طوری کامل کنید که مقدار پول را به عنوان ورودی تابع بگیرد و تعداد حالات مذکور را به عنوان خروجی برگرداند.

```
int NumberOfWays(int n) {
```

```
}
```

(مثال)

input = 14 output = 0

input = 16 output = 1 → { 2 سکه 2 تومانی، 1 سکه 5 تومانی، 1 سکه 7 تومانی }

input = 50 output = 6

{ 12 سکه 2 تومانی، 1 سکه 5 تومانی، 3 سکه 7 تومانی }

{ 6 سکه 2 تومانی، 2 سکه 5 تومانی، 4 سکه 7 تومانی }

{ 14 سکه 2 تومانی، 3 سکه 5 تومانی، 1 سکه 7 تومانی }

{ 8 سکه 2 تومانی، 4 سکه 5 تومانی، 2 سکه 7 تومانی }

{ 2 سکه 2 تومانی، 5 سکه 5 تومانی، 3 سکه 7 تومانی }

{ 4 سکه 2 تومانی، 7 سکه 5 تومانی، 1 سکه 7 تومانی }

پاسخ: ما باید ببینیم این معادله زیر به ازای a و b و c های مختلف، چند جواب معتبر دارد.

$$7a + 5b + 2c = n \quad (a, b, c \geq 1) \wedge (c = 2k, k \in \mathbb{N})$$

با هر بار حالت بندی روی متغیرهای a و b و c ، چک میکنیم که آیا شرایط برقرار است یا خیر.

شبه کد قسمت الف)

```
initialize temp, count = 0
```

```
initialize a, b=1, c
```

```
while b * 5 < n then
```

```
    a = 1
```

```
    while (a * 7) + (b * 5) < n then
```

```
        temp = n - (a * 7) - (b * 5)
```

```
        if temp MOD 2 == 0 then
```

```
            c = temp Div 2
```

```
            if (c MOD 2 == 0) and (c != 0) then
```

```
                count = count + 1
```

```
            endif
```

```
        endif
```

```
        a = a + 1
```

```
    endwhile
```

```
    b = b + 1
```

```
endwhile
```

```
int NumberOfWays(int n) {  
    int count = 0;  
  
    for(int b = 1; (5 * b) < n; b++){  
        for(int a = 1; (7 * a) + (5 * b) < n; a++){  
            int m = n - ((7 * a) + (5 * b));  
  
            if(m % 2 == 0){  
                int c = m / 2;  
  
                if((c % 2 == 0) && (c != 0)){  
                    count++;  
                }  
            }  
        }  
    }  
  
    return count;  
}
```